Robert Louis Bostick

CYBERSECURITY-FOCUSED COMPUTER SCIENCE STUDENT

1030 east Franklin st 76645 | Robertlouisbostick@yahoo.com | 2544793842



PROFESSIONAL SUMMARY

Computer Science student with a cybersecurity focus and hands-on experience building browser-based security simulations, indicator-of-compromise workflows, and technical demonstrations. Applies programming, threat-analysis concepts, and structured testing to make security logic visible, repeatable, and explainable. Brings a practical learning style grounded in building, documenting, testing, and refining working systems.

CORE CAPABILITIES

Cybersecurity: IOC analysis, threat hunting concepts, detection logic, log analysis, MITRE ATT&CK mapping, penetration-testing labs, sandboxed experimentation

Development: Python, C++, JavaScript, HTML, CSS, single-file browser applications, structured JSON output

Platforms and tools: Windows, Windows Sandbox, Hack The Box, browser developer tools, Chart.js, Tailwind CSS, AI-assisted development workflows

Working methods: requirements breakdown, iterative prototyping, threshold testing, evidence capture, technical documentation, honest limitation analysis

EDUCATION

CTU Online
Bachelor's in Computer Science, Cybersecurity Focus
Junior standing; 123 of 183 credits completed

SELECTED PROJECT EXPERIENCE

Aegis Shield v2.5 - Mobile IOC SOC Simulator

Independent project

HTML, CSS, JavaScript, Chart.js, Tailwind CSS, JSON

- Designed a mobile-first Security Operations Center simulator with baseline, thunderstorm, and ransomware scenarios.
- Implemented a ten-second sliding-window detector with tunable packet-rate and loss-burst thresholds.
- Created deduplicated alert paths for loss bursts, SYN floods, sustained packet surges, and synthetic ransomware behavior.
- Mapped defensible findings to MITRE ATT&CK techniques and documented unimplemented or simulated states instead of overstating capability.
- Added live telemetry visualization, terminal-style evidence, status transitions, optional audio feedback, and downloadable JSON audit reports.

File Carving Arcade

Independent project

Portable browser-based cybersecurity learning tool

- Created a portable browser-based version of a file-carving project for hands-on learning and demonstration.
- Focused the experience on practical interaction, portability, and clear presentation of technical concepts.

Robert Louis Bostick

CYBERSECURITY PROJECT EXPERIENCE

Robertlouisbostick@yahoo.com | 2544793842

CYBERSECURITY LAB AND ANALYSIS WORK

IOC Monitoring and Threat-Analysis Development

Ongoing lab work

- Builds and evaluates indicator-of-compromise monitoring concepts using structured events, alert thresholds, and analyst-facing outputs.
- Explores real-time log streaming and event correlation approaches for cybersecurity monitoring systems.
- Organizes technical evidence from multiple AI-assisted development sessions into a physical demonstration portfolio.
- Uses repeatable scenarios and visible decision points to explain why an alert should or should not fire.

Penetration Testing and Defensive Security Practice

Independent study

- Uses Hack The Box and Windows-based lab environments to practice cybersecurity concepts and controlled testing.
- Works within sandboxed and educational environments when exploring penetration-testing tools and techniques.
- Studies threat hunting, IOC analysis, cryptology, and security operations through project-based learning.

ADDITIONAL TECHNICAL DEVELOPMENT

- Develops Python-based cybersecurity and automation concepts and uses C++ for structured programming work.
- Builds interactive simulations and games with state, branching logic, save/load concepts, and user-facing feedback.
- Experiments with multi-system AI-assisted workflows while retaining human review, iterative testing, and documentation.
- Explores Meta Quest 3 development and immersive interfaces as an additional technical interest.

PROFESSIONAL STRENGTHS

- **Systems thinking:** breaks complex ideas into components, interfaces, rules, and evidence.
- **Practical learning:** converts coursework and research into working prototypes.
- **Documentation:** explains architecture, test conditions, results, limitations, and next steps.
- **Iteration:** revises designs based on observed behavior rather than treating the first build as final.
- **Integrity:** distinguishes simulated capability from production capability and keeps technical claims evidence-based.

INCLUDED PORTFOLIO EVIDENCE

The following Aegis Shield case study includes architecture, detection logic, captured baseline and attack-scenario results, MITRE ATT&CK mapping, a verification matrix, limitations, and an engineering roadmap.

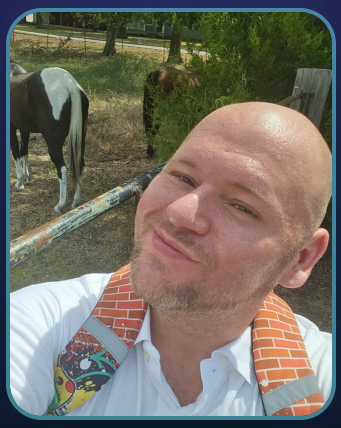
> **Featured result:** at a 500 packets/second threshold and a five-second loss threshold, the baseline produced zero alerts; the storm scenario produced three distinct alerts; and the ransomware scenario produced one consolidated encryption-behavior alert.

CYBERSECURITY ENGINEERING PORTFOLIO

Robert Louis Bostick

AEGIS SHIELD v2.5

Mobile security operations simulator for explainable alerting, threshold tuning, MITRE ATT&CK alignment, and auditable output.



Robert Louis Bostick

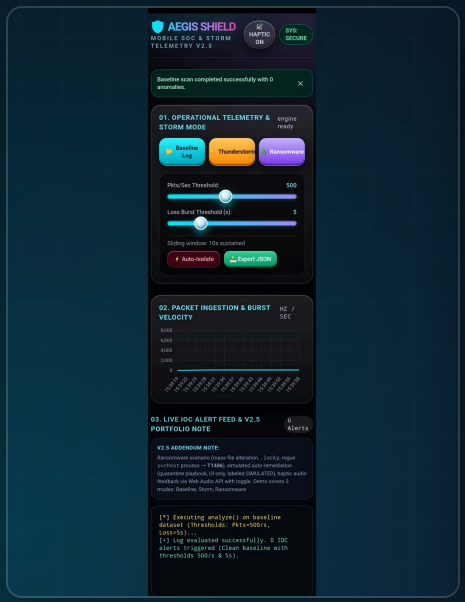
1030 east Franklin st 76645
Robertlouisbostick@yahoo.com | 2544793842
CTU Online - Bachelor's in Computer Science, Cybersecurity Focus - 123 of 183 credits completed

CASE STUDY

Design goal: translate raw synthetic telemetry into visible, testable decisions without requiring a full SIEM deployment.

EVIDENCE SET

Baseline, thunderstorm, and ransomware scenarios captured from the working mobile interface.







EXECUTIVE BRIEF

A compact prototype makes detection logic visible

Aegis Shield is a single-file, mobile-first Security Operations Center (SOC) simulator. It demonstrates how event streams can be normalized, evaluated against transparent rules, presented to an analyst, and exported as an audit record.

3
SIMULATION MODES
Baseline, thunderstorm, and ransomware.

10s
SLIDING WINDOW
Sustained packet-rate evaluation.

JSON
AUDIT OUTPUT
Mode, thresholds, alerts, timestamp.

What I built

- A responsive HTML/CSS interface optimized for touch.
- A JavaScript event generator for repeatable synthetic scenarios.
- A queue-based ten-second packet window with tunable thresholds.
- Deduplicated alert classes for loss, SYN-flood, packet-rate, and ransomware behavior.
- Live charting, status transitions, audio feedback, ATT&CK labels, and JSON export.

What this demonstrates

- Turning detection requirements into deterministic logic.
- Separating clean traffic, environmental interference, and hostile behavior.
- Designing for analyst readability on a small screen.
- Documenting limitations instead of overstating prototype capability.
- Connecting engineering output to an auditable review workflow.

Portfolio position: This is an explainable simulation and teaching tool, not a production intrusion detection system. Its value is the visible chain from generated event to rule decision to analyst evidence.

SYSTEM DESIGN

The pipeline keeps each decision inspectable

The implementation is intentionally compact: one browser document contains the interface, scenario generator, analysis engine, and report export. Each stage exposes enough state for a reviewer to trace why an alert fired.

1. Generate
Create timestamped packet, loss, SYN-flood, or file-lock events.

2. Normalize
Read event type, count, timestamp, and detail fields.

3. Window
Maintain packet timestamps inside a ten-second queue.

4. Detect
Evaluate thresholds and signature-like events once per run.

5. Present
Update chart, terminal, status, ATT&CK table, and JSON report.

Core rule behavior

```
windowMs = 10 * 1000
prune(packetQueue, olderThan = windowMs)

if queueLength >= packetThreshold * 10:
  alert("POSSIBLE_DDOS")

if consecutiveLoss >= lossThreshold:
  alert("LOSS_BURST")

if event == "syn_flood":
  alert("SYN_FLOOD")

if event == "crypto_lock" and not seen:
  alert("RANSOMWARE_ENCRYPTION")
```

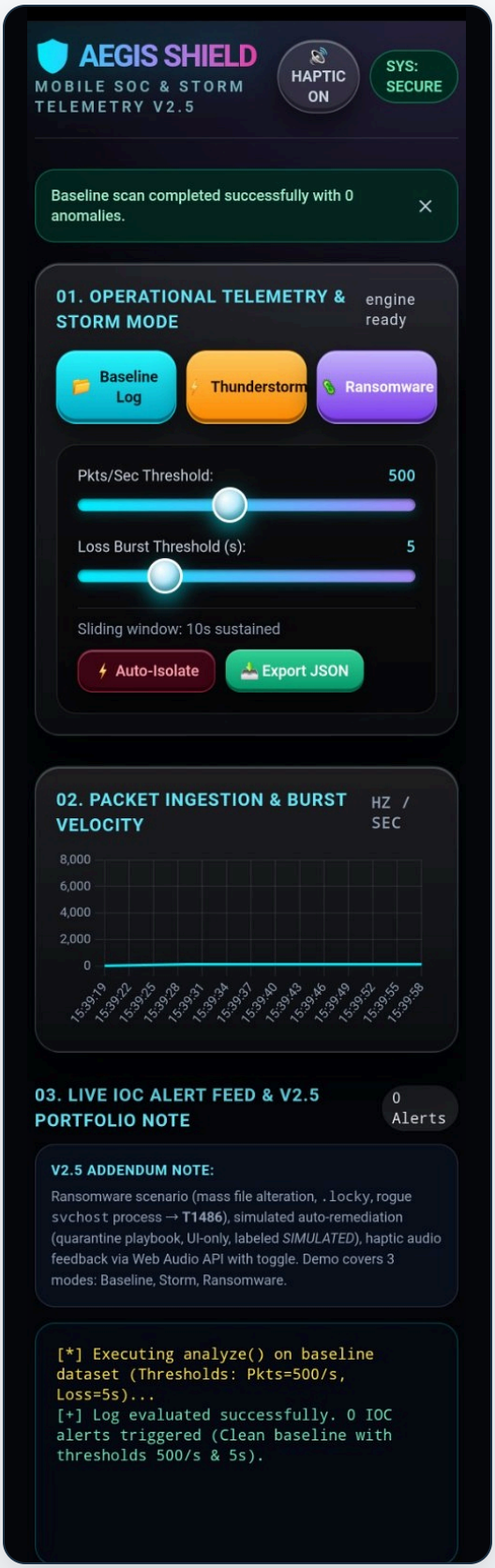
Engineering controls

- ✓ **Threshold inputs:** packet rate from 100-1,000 packets/second; loss burst from 2-15 seconds.
- ✓ **Alert suppression:** Boolean run-state prevents repeated DDoS, loss, and ransomware alerts.
- ✓ **Loss handling:** packet queue resets during simulated heavy loss.
- ✓ **Traceability:** terminal output records timestamp, alert class, evidence, and threshold.
- ✓ **Exportability:** the browser downloads a structured JSON record after each run.
- ✓ **Mobile feedback:** touch-scale controls and Web Audio cues support rapid interaction.

> **Boundary:** “Auto-Isolate” is a simulated response control. In v2.5 it changes interface state and writes a playbook message; it does not quarantine a real host or terminate a process.

SCENARIO EVIDENCE 01

The clean baseline stays quiet



Observed result

0

ALERTS

The interface reports **SYS: SECURE** after evaluating the baseline dataset.

Test configuration

Packet threshold	`500/s`
Loss threshold	`5s`
Window	`10s`
Input pattern	40 packet events at a count of 12 each

Why the result matters

The normal event stream never reaches the sustained packet-rate threshold and contains no consecutive loss, SYN-flood, or file-lock event. The detector therefore produces no alert instead of manufacturing activity.

- ✓ Threshold values match the terminal record.
- ✓ The chart remains near the floor of the configured scale.
- ✓ The export control becomes available after analysis.

> **Decision:** baseline behavior passes the negative-control test.

SCENARIO EVIDENCE 02

The storm run separates three alert conditions



Observed result

3

DISTINCT ALERTS

The same 500/s and 5s configuration produces loss, SYN-flood, and sustained-rate findings.

ALERT	EVIDENCE SHOWN
LOSS_BURST	Five consecutive packet-drop seconds met the configured loss threshold.
SYN_FLOOD	The synthetic event carries a distributed-botnet signature.
POSSIBLE_DDOS	5,335 packets in the ten-second window exceeded the 5,000-packet rule boundary.

What the graph adds

The telemetry curve shows a low-volume period followed by a steep burst near 6,000 queued packets. The terminal preserves the exact trigger count, allowing visual inspection and rule-level verification to agree.

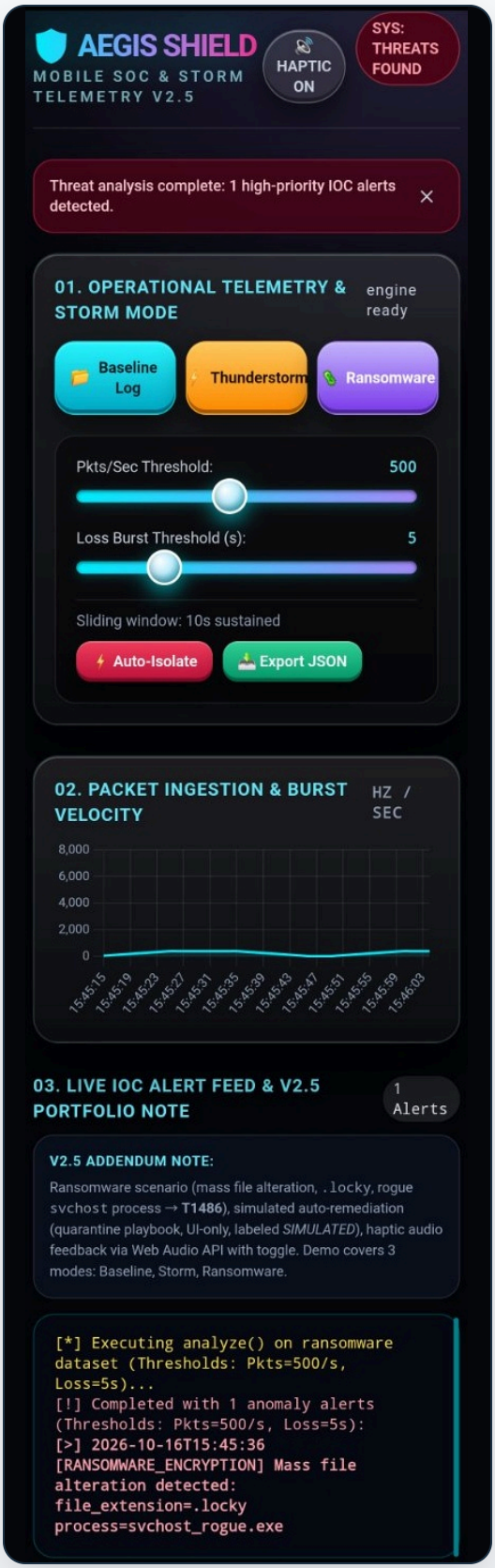
Noise control

Each threshold-driven class fires once per simulation run. This avoids flooding the analyst feed while retaining the first threshold crossing.

> **Decision:** storm behavior passes the positive-control test for all three configured conditions.

SCENARIO EVIDENCE 03

The ransomware rule is independent of traffic volume



Observed result

1

RANSOMWARE ALERT

The event detector identifies mass file alteration and reports `RANSOMWARE_ENCRYPTION`.

Detection evidence

Event type	`crypto_lock`
Extension marker	`.locky`
Process marker	`svchost_rogue.exe`
ATT&CK alignment	**T1486** - Data Encrypted for Impact

Why one alert is correct

The scenario generates repeated file-lock events from seconds 21 through 34, but the run-state flag records one consolidated ransomware alert. That preserves signal without printing the same finding fourteen times.

> **Decision:** signature-like behavior remains detectable even when the packet-rate and loss-burst rules are not crossed.

> **Scope:** the event names and process markers are synthetic. The prototype does not inspect a live file system or validate cryptographic activity.

ATT&CK ALIGNMENT

The strongest mappings are explicit; the gaps stay visible

The interface references four MITRE ATT&CK techniques. The table below distinguishes implemented evidence from conceptual display states so the prototype is not presented as more mature than it is.

ID	TECHNIQUE	PROTOTYPE EVIDENCE	ASSESSMENT
T1498	Network Denial of Service	Sustained packet volume crosses the ten-second rate threshold.	**Defensible mapping**
T1486	Data Encrypted for Impact	A synthetic file-lock event carries extension and process markers.	**Defensible simulation**
T1499	Endpoint Denial of Service	The interface displays “Mitigated,” but no endpoint-resource detector or enforcement action is implemented.	**Prototype label only**
T1046	Network Service Discovery	The standby table includes the technique; the supplied event engine has no scan or service-discovery rule.	**Not implemented**

Accurate claim

Aegis Shield demonstrates rule design, alert presentation, scenario testing, and ATT&CK-informed labeling against synthetic input.

Claim deliberately avoided

It does not ingest production telemetry, prove a real endpoint was isolated, or establish operational detection coverage.

Recommended revision: drive the ATT&CK table from the alert list instead of replacing it with a fixed threat-state table. Rename “Mitigated” to “Response simulated” until containment is connected and tested.

Reference basis

MITRE ATT&CK Enterprise technique pages: T1498, T1499, T1486, and T1046. Accessed October 16, 2026.

REVIEW GUIDE

A sixty-second demonstration proves the full chain

1. Establish the negative control
Set 500 packets/second and 5 seconds. Run Baseline. Confirm zero alerts and secure state.

2. Exercise the storm rules
Run Thunderstorm. Confirm one loss alert, one SYN-flood alert, and one sustained-rate alert.

3. Exercise the file-impact rule
Run Ransomware. Confirm one mass-file-alteration alert and T1486 alignment.

4. Preserve the evidence
Export the JSON record and explain that containment remains simulated.

Verification matrix

TEST	EXPECTED	OBSERVED IN EVIDENCE	RESULT
Baseline, 500/s, 5s	No alert	0 alerts; secure state	Pass
Storm loss burst	One loss alert	Five-second threshold reached	Pass
Storm SYN event	One signature alert	Distributed-botnet detail recorded	Pass
Storm packet surge	One rate alert	5,335 packets in window	Pass
Repeated file-lock events	One ransomware alert	Consolidated alert emitted	Pass
Auto-Isolate	UI demonstration only	No real host action	Scoped

Current limitations

- Synthetic events only; no packet capture or endpoint sensor.
- Tailwind CSS and Chart.js load from content delivery networks.
- No authentication, persistent case storage, or integrity signing.
- Browser-side timestamps and data remain local to the demo session.

Next engineering increments

- Accept replayed PCAP-derived or structured log input.
- Add a real T1046 discovery scenario before displaying that mapping.
- Make response state evidence-driven and explicitly simulated.
- Add test fixtures for threshold boundaries and alert deduplication.
- Package dependencies locally and hash exported reports.

Bottom line: Aegis Shield v2.5 is portfolio-ready as an honest, inspectable prototype. Its strongest evidence is not the neon interface; it is the repeatable relationship between input, threshold, alert, and exported record.